

EXPERIMENT NO: 05

Name: Shreya Sawant

Class: D15A

Roll No: 53

Aim: To implement navigation, routing, and gestures in a Flutter application

Theory:

Navigation & Routing in Flutter

Navigation is the process of moving between different screens (or pages) in an app. Since Flutter follows a **widget-based architecture**, each screen is represented as a widget, and navigation is handled using a **stack-based system** (similar to how web browsers manage page history).

Navigation Approaches in Flutter:

1. **Imperative Navigation (Push-Pop Model)**
 - Uses `Navigator.push()` to open a new screen.
 - Uses `Navigator.pop()` to return to the previous screen.
 - Works like a stack (Last In, First Out - LIFO).
2. **Declarative Navigation (Named Routes & GoRouter)**
 - Uses predefined route names to navigate.
 - Helps in managing complex app navigation efficiently.
3. **Navigation with State Management**
 - Used in large-scale applications where navigation state is maintained using providers like **Provider, Riverpod, Bloc, or GetX**.
4. **Deep Linking & Dynamic Routing**
 - Allows external URLs to open specific screens within the app.
 - Useful for handling notifications and web links.

Routing in Flutter

Routing determines how users navigate through different sections of an application. It enables:

- **Defining and managing screens efficiently.**
- **Enhancing UX by ensuring smooth transitions.**

- **Passing and receiving data between screens.**

Types of Routing:

- **Basic Routing:** Manually navigating between screens using the Navigator class.
- **Named Routing:** Using a routes map to define screen names and their respective widgets.
- **On-Generate Routing:** Dynamically controlling navigation flow using conditions.
- **Nested Navigation:** Managing multiple navigation flows within tabs or bottom navigation bars.

Gestures in Flutter

Gestures allow users to interact with the app through touch-based actions, providing a more engaging user experience. Flutter has a powerful **GestureDetector** widget that helps recognize and handle different gestures.

Common Gesture Types:

- **Tap Gesture:** Used for buttons, links, and UI interactions.
- **Double Tap:** Often used for liking content or zooming.
- **Long Press:** Triggers additional options or context menus.
- **Swipe & Drag:** Used in carousels, sliders, and scrollable content.
- **Pinch & Zoom:** Common in images and maps.

Gesture Handling Techniques:

- **Using GestureDetector:** To detect gestures manually.
- **Using InkWell or InkResponse:** Provides visual feedback for tappable elements.
- **Custom Gesture Recognition:** Flutter allows combining multiple gestures using GestureRecognizer.

Importance of Navigation, Routing, and Gestures in Flutter Apps

- **User-Friendly Experience** – Navigation helps users seamlessly explore the app.
- **Efficient State Management** – Proper routing keeps the app organized.
- **Enhanced Interactivity** – Gestures make the app feel intuitive and engaging.
- **Optimized Performance** – Declarative navigation and lazy loading improve performance.
- **Platform Consistency** – Flutter's navigation system works smoothly on Android, iOS, and the web.

Code Snippet: main_navigation.dart

```
import 'package:flutter/material.dart';
import 'package:etsy_clone/theme.dart';
import 'home_screen.dart';
import 'deals_screen.dart';
import 'shop_screen.dart';
import 'favorites_screen.dart';
import 'cart_screen.dart';

void main() {
  runApp(EtsyCloneApp());
}

class EtsyCloneApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: MainNavigation(),
    );
  }
}

class MainNavigation extends StatefulWidget {
  @override
  _MainNavigationState createState() => _MainNavigationState();
}

class _MainNavigationState extends State<MainNavigation> {
  int _selectedIndex = 0;

  final List<Widget> _screens = [
    HomeScreen(),
    DealsPage(),
    ShopPage(),
    FavoritesScreen(favoriteItems: []),
    CartScreen(),
  ];

  void _onItemTapped(int index) {
```

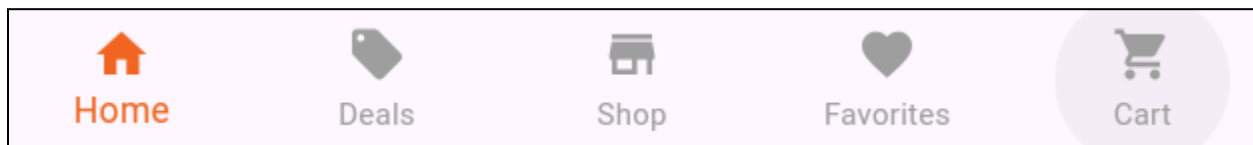
```

setState() {
  _selectedIndex = index;
});
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    body: _screens[_selectedIndex],
    bottomNavigationBar: BottomNavigationBar(
      type: BottomNavigationBarType.fixed,
      selectedItemColor: AppColors.primary,
      unselectedItemColor: AppColors.textSecondary,
      currentIndex: _selectedIndex,
      onTap: _onItemTapped,
      items: const [
        BottomNavigationBarItem(icon: Icon(Icons.home), label: "Home"),
        BottomNavigationBarItem(icon: Icon(Icons.local_offer), label: "Deals"),
        BottomNavigationBarItem(icon: Icon(Icons.store), label: "Shop"),
        BottomNavigationBarItem(icon: Icon(Icons.favorite), label: "Favorites"),
        BottomNavigationBarItem(icon: Icon(Icons.shopping_cart), label: "Cart"),
      ],
    ),
  );
}

```

This main_navigation.dart file is responsible for navigation between pages from the bottom navigation bar



Navigation Routing

- Splash screen to welcome screen

```
Future.delayed(Duration(seconds: 3), () {  
  Navigator.pushReplacement(  
    context,  
    MaterialPageRoute(builder: (context) => WelcomeScreen()),  
  );  
});
```

- Banner tile in home screen to cart screen

```
ElevatedButton(  
  onPressed: () {  
    Navigator.push(  
      context,  
      MaterialPageRoute(builder: (context) => CartScreen()),  
    );  
  },  
  child: Text("Shop Now", style: TextStyle(color: Colors.white)),  
);
```

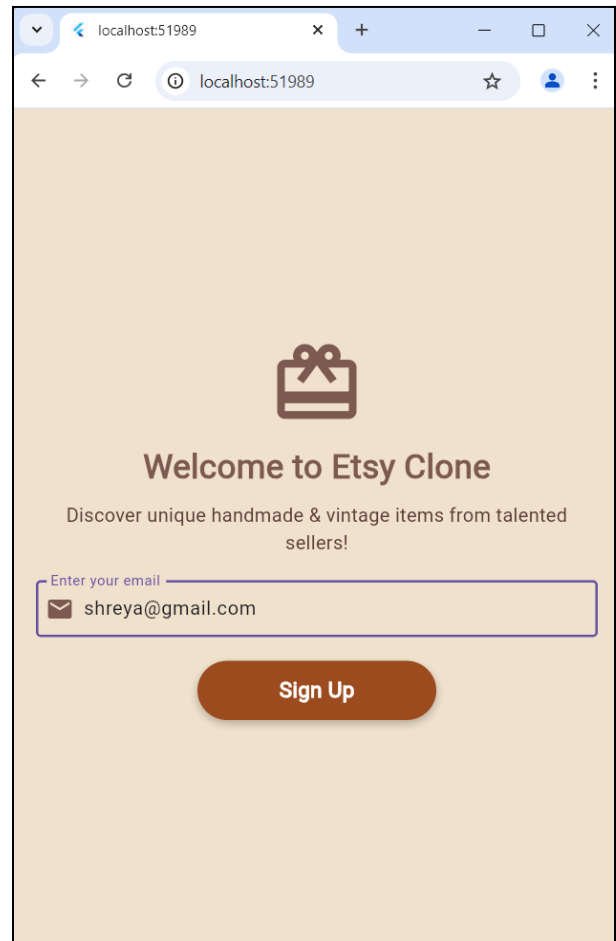
Gesture Detection

- Search bar

```
GestureDetector(  
  onTap: () {  
    setState(() {  
      isExpanded = true;  
    });  
  },  
  child: AnimatedContainer(  
    // Container properties  
  ),  
);
```

The app opens with a splash screen, then moves to the welcome screen. Clicking "Sign Up" takes the user to the sign-up page, and after registering, they reach the home screen. Here, a "Shop Now" button leads to the cart, where users can review their items and billing details. Finally, clicking "Proceed to Checkout" takes them to a form to complete their purchase.

Screenshot:



localhost:51989

localhost:51989

←

→

↻

🔒

☆

👤

← Sign In to Etsy

👤 shreya@gmail.com

[Use a different email address](#)

Create your account

Registration is easy.

First name *

Password *

● At least 8 characters

● At least one uppercase letter

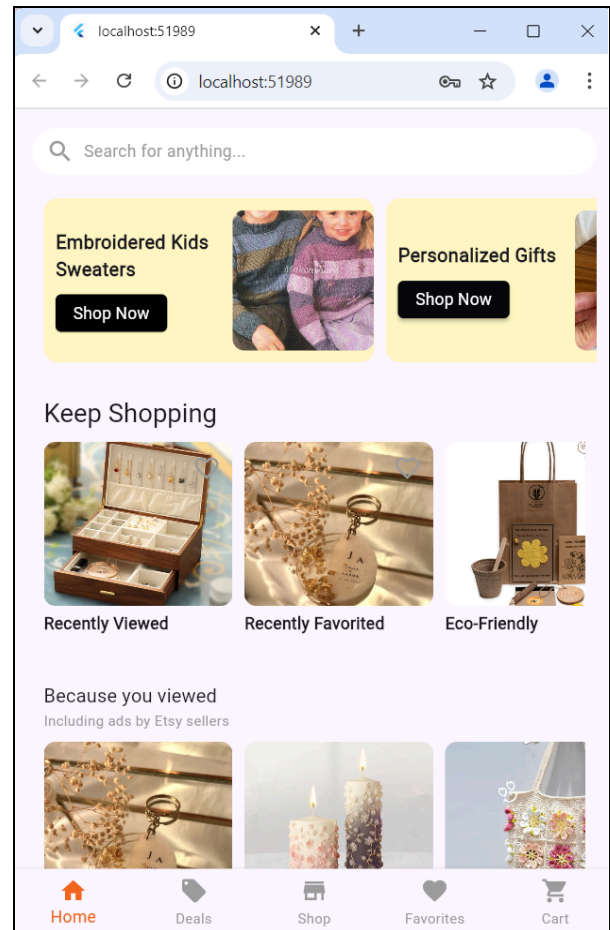
● At least one number

● At least one special character

By clicking Register, you agree to Etsy's [Terms of Use](#) and [Privacy Policy](#).

Register

Etsy may send you communications; you may change your preferences in your account settings. We'll never post without your permission.



localhost:51989

localhost:51989

←

→

↺

🔍

☆

👤

⋮

←

Your Cart



Clothing Item
₹500

− 1 +



Resin Keychain
₹250

− 1 +



Stylish Bag
₹800

− 1 +

Mark order as a gift

☐

Enter Coupon Code

✓

Subtotal:

₹1550

Shipping:

₹100

Tax (18% GST):

₹279.00

Total Price:

₹1929.00

Proceed to Checkout

localhost:58875

localhost:58875

←

→

↺

🔍

☆

👤

⋮

←

Checkout

Full Name

Phone Number

Address

City

State

PIN Code

Payment Method

Card

Personalized Message (Optional)

Quantity

1

Place Order